

[Open in app](#)

Data Science Colle... · Following

 Member-only story

Building conversational analytics tool with Langchain and Qwen3 Coder : Hands on guide with lessons

Learnings from a weekend project to create an intelligent data analysis agent that can understand natural language queries and generate insights autonomously

16 min read · Sep 7, 2025



Prateek Bhatnagar

 Listen Share More

By far one of the most common AI agents use case is data analysis with prompts. Many ecosystems such as databricks (Databricks Genie) , Azure(Azure AI Foundry/Azure OpenAI) , AWS (Amazon Q) offer this as a scalable capability within their ecosystem other than OpenAI , Perplexity and other services that also allow text to analytics with data attachments . This example is built up for self learning and explores building one's own app with an integrated data analyst to answer all your data queries via LLM generated code.

This project had a lot of learnings despite a lot of material and examples online which will be covered in the end.

Which framework?

We will use Langchain and within it langchain-experimental which has a series of agent to run python codes. Agent is a class that uses an LLM to choose a sequence of actions to take. In Chains, a sequence of actions is hardcoded. In Agents, a language model is used as a reasoning engine to determine which actions to take and in

which order. Agents select and use Tools and Toolkits for actions. These tools are defined by the user. We can also natively build this using Python REPL as the tool but for faster prototyping a existing agent capability is used. Linking an LLM to a Python REPL, makes the task complex with the orchestration aspect but for more control over aspects such as cache, security and customizability — one can build it from scratch.

Use of Create_pandas_dataframe_agent

This is a specialized agent builder that wires an LLM to analyze one or more Pandas DataFrames by iteratively writing and running Python code behind the scenes, typically via a Python REPL (Read-Eval-Print Loop) tool. Designed for DataFrame-centric analysis (filters, groupby, joins, correlations, plotting), with options like `include_df_in_prompt`, `multiple_dataframes`, and `modin_engine`. It runs on the host environment (your VS Code session), so it has full access to your local Python, files, and installed packages . One has to use with caution due to security risks. Good for generic code execution, calculations, and ad-hoc data work when an agent needs raw Python execution capabilities. I would not recommend this for complex analysis , pipelines , ML workflows etc.

Why?

`create_pandas_dataframe_agent` adds planning, guardrails, and dataframe-specific “skills” on top of raw Python execution, so you don’t have to handhold the model with every step or worry as much about brittle prompting.

Beyond the LLM prompt + Python REPL, it takes care of :

- **DataFrame-oriented prompting and heuristics:** It’s purpose-built for pandas tasks (inspect head, infer column types, pick operations like `groupby/merge/plot`) rather than generic code execution, reducing prompt engineering you’d otherwise do to make the model reliably choose the right pandas idioms.
- **Iterative tool use loop:** It maintains context across steps (e.g., run code, examine outputs/exceptions, refine the next snippet), which you’d otherwise have to implement yourself with custom controller logic.
- **Safer defaults for DF access:** It constrains the problem-space to the provided DataFrame(s) and typical pandas operations, lowering chances of the LLM drifting into unrelated code if your prompt is underspecified.

- **Built-in options:** Things like including sample rows in the prompt, handling multiple DataFrames, setting max iterations, and plugging in extras, so you don't have to rebuild these knobs yourself.

Better reliability for non-expert prompts: If the user says “compare average revenue by region and plot the top 5,” it's more likely to autonomously produce the correct pandas code, read intermediate results, and fix small errors without you writing a long, brittle system prompt.

Other considerations

BearlyInterpreterTool — A tool that executes Python in an isolated sandbox via the Bearly service; supports popular libraries (pandas, numpy, matplotlib), file upload into the sandbox, and saving outputs to output/ with links returned. Sandbox resets every run, so each script must be self-contained; you can add files to the sandbox and generate images/tables via saved files. Much safer for untrusted code because it's remote and isolated, but it's not your local VS Code Python environment.

E2B Data Analysis A Lang Chain tool that connects to E2B's cloud “Data Analysis” sandbox with Python, matplotlib, dynamic pip/system package installs, shell commands, and file up/download; built explicitly for safe LLM runtime sandboxes and “Advanced Data Analysis” experiences.

Getting started

Below example first focuses on building a skeleton of the agent that can perform data analysis and then later integrate this backend with Streamlit's intuitive interface. We'll create a system that can intelligently classify user queries, execute appropriate analysis code, and present results in a polished format.

1. Loading the libraries

```
# load base packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import langchain as lc
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
import anyio
from langchain_openai import OpenAI
from langchain_experimental.agents import create_pandas_dataframe_agent
```

```
import seaborn as sns
from langchain.schema import SystemMessage, HumanMessage
import logging
from typing import Any, Optional
from langchain_experimental.agents.agent_toolkits import create_pandas_dataframe_agent
import warnings
import io
import base64
from langchain.output_parsers import OutputFixingParser
from langchain_core.exceptions import OutputParserException
from langchain_core.output_parsers import BaseOutputParser
import time
```

2. Setting up LLM API

We use <https://openrouter.ai/> for LLMs. You can set up your `.env` file in your working folder such as given below :

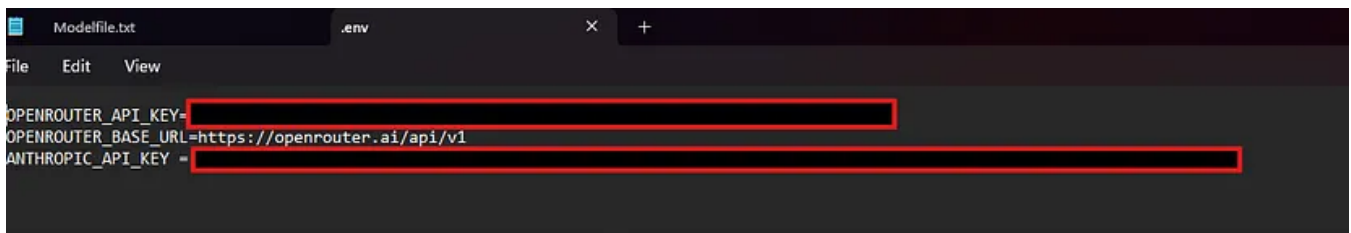


Image by Author

They have all range of models there and one can use any basis coding capabilities, latency and cost per million tokens. For our example — we use Qwen coding model, <https://openrouter.ai/qwen/qwen3-coder>. There are some free models as well but there is a per minute and token limit. It is good for initial testing of API but you would have to top up with some credits for further build of this app. Once you have the API key and URL, save them `.env` in your working folder.

You can now set your environment variables

```
load_dotenv() # This loads variables from .env

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = os.getenv("OPENROUTER_BASE_URL")
```

3. Setting up

Now we write a function that returns a chat client that speaks the OpenAI-style chat/completions API. Although it's named ChatOpenAI because it implements that interface, not because it's locked to OpenAI's servers. We let the user provide the model name and create multiple such clients for testing.

```
def create_chat_openai(model):  
    """  
    Create a ChatOpenAI instance with a specified model.  
  
    Args:  
        model (str): The model to use (e.g., "meta-llama/llama-3.1-405b-instruct")  
  
    Returns:  
        ChatOpenAI: A ChatOpenAI instance with the specified model.  
    """  
    try:  
        llm = ChatOpenAI(  
            model=model,  
            api_key=OPENROUTER_API_KEY,          # from your .env  
            api_base=OPENROUTER_BASE_URL,        # required for OpenRouter  
            #messages=[SystemMessage(content=instructions)],  
            streaming=False,  
            verbose=False,  
            temperature=0.0, # Set to 0 for deterministic output  
            max_retries=3,  
        )  
        return llm  
    except Exception as e:  
        ...
```

Now let's explore some models. We will set up two clients, one being for initial testing and available for free with limits on OpenRouter. Llama 405B model is well known.

Meta: Llama 3.1 405B Instruct (free) 📁

meta-llama/llama-3.1-405b-instruct:free 📁

Image by Author

```
# define the clients
llm_test = create_chat_openai(model = "meta-llama/llama-3.1-405b-instruct:fre
llm = create_chat_openai(model = "qwen/qwen3-coder") # final coding model for a
```

Now we do a simple test using langchain's invoke method — `llm.invoke()` is a method to interact with Large Language Models (LLMs). It represents a synchronous call to the LLM, sending a single input (typically a prompt) and receiving a complete response.

A screenshot of a Jupyter Notebook interface. The top part shows a code cell with the following Python code:

```
prompt = "What is the capital of France?"
response = llm_test.invoke(prompt)
print(response.content)
```

 Below the code, the output is displayed:

```
[8]: ✓ 8.5s
... The capital of France is Paris.
```

 The notebook has a dark theme and a toolbar at the top right.

Image by Author

4. Initial build of conversational analytics

4.1 Getting data ready

First we start with setting up the ever classic `titanic` dataset that has been a solid companion dataset for millions of prototypes and reference data for reproducible codes.

```
# Load Titanic dataset
df = sns.load_dataset("titanic")
```

4.2 Setting up the agent

In this we pass the data frame created above and `llm` instance using Qwen

```
agent = create_pandas_dataframe_agent(
    llm,
    df,
    verbose=False,
    allow_dangerous_code=True,
```

```
        return_intermediate_steps=False,  
    )
```

4.3 Defining Helper functions

Now before we use the agent — we need good to ensure that :

- LLM is able to understand the request and format of the output which for simplicity can be classified as a statistic, table or graph. We will write a prompt classifier function using llm so that the user does not have to follow a strict prompt template. This is essential to limit the outputparser issues from the LLM which we will discuss in more details in later part of this article.

```
# Define classification prompt as a constant  
CLASSIFICATION_PROMPT = """  
You are a classifier. Read the user request carefully and decide what type of c  
There are ONLY three possible categories: `statistic`, `table`, or `graph`.  
RULES:  
- The answer MUST be strictly one word: either `statistic`, `table`, or `graph`  
- Do not explain. Do not output anything else.  
- Example: User request: "generate a bar plot of gender" -> Answer: graph  
- Example: User request: "show me the correlation matrix" -> Answer: table  
- Example: User request: "what is the average age" -> Answer: statistic  
"""  
  
# User query  
user_query = "generate a bar plot of gender"  
  
# get classification result  
classification_result = llm.invoke(CLASSIFICATION_PROMPT + f"\nUser request: '{  
prompt_type = classification_result.content.strip().lower()
```

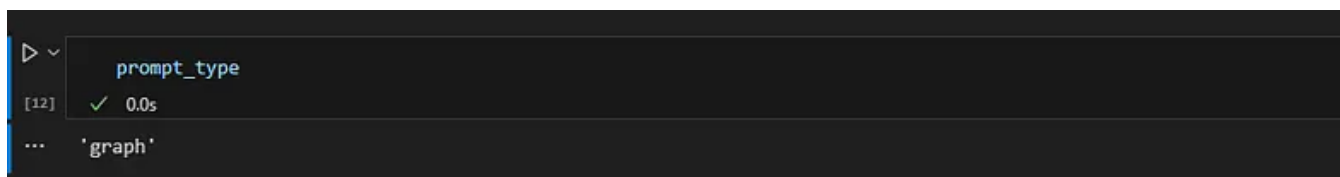
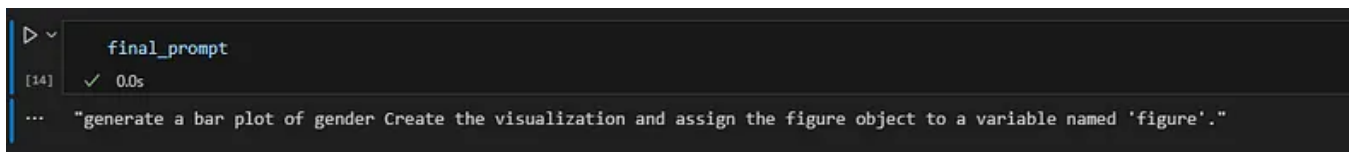


Image by author

- Now that LLM has understood the nature of prompt, we will specify the prompt going to the agent depending on the output type retrieved above and retrieve the outputs. Once these outputs are generated, they may be printed however to access these variables they exist inside the tool/agent call unless explicitly returned and captured, or stored in a user-managed variable or memory mechanism that is coded to hold them. That is precisely what we mention in this prompt and retrieve it thereafter

```
# Step 2: Formulate the final prompt based on the classification
prompt_map = {
    "statistic": f"{user_query} Assign the final result value to a variable named 'result'."
    "table": f"{user_query} Assign the output dataframe to a variable named 'table'."
    "graph": f"{user_query} Create the visualization and assign the figure object to a variable named 'figure'."
}
final_prompt = prompt_map.get(prompt_type, user_query)
```



```
final_prompt
[14] ✓ 0.0s
... "generate a bar plot of gender Create the visualization and assign the figure object to a variable named 'figure'."
```

Image by Author

- Testing our user_query and final_prompt


```
# Invoke the agent
agent_response = agent.invoke({"input": final_prompt})
```

✓ 4.5s

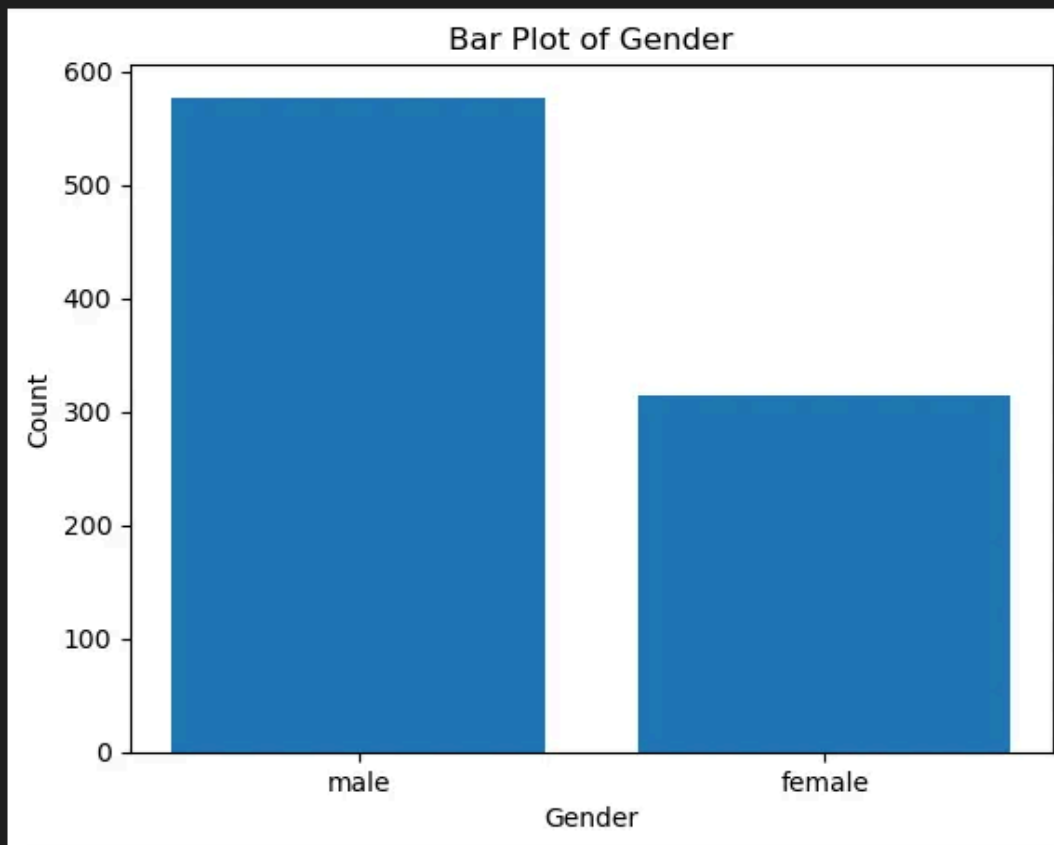


Image by Author

- Retrieving and saving the variable outputs from locals

```
def get_agent_variable(agent, variable_name: str):
    """
    Retrieves a variable from the agent's Python tool local execution environment.

    Args:
        agent: The pandas dataframe agent instance.
        variable_name (str): The name of the variable to retrieve (e.g., 'temp_').

    Returns:
        The variable's value if found, otherwise None.
    """
    try:
        # The Python REPL tool is typically the first tool in this agent
        python_tool = agent.tools[0]

        # Access the dictionary of local variables from the tool's state
        locals_dict = python_tool.locals
```

```
# Safely get the variable by name; returns None if not found
return locals_dict.get(variable_name)
except (IndexError, AttributeError) as e:
    print(f"Error: Could not access tools or local variables from the agent")
    return None
```

```
# Retrieve the result from agent's local variables
variable_map = {
    "statistic": "sg_value",
    "table": "temp_df",
    "graph": "figure",
}
variable_name = variable_map.get(prompt_type)
raw_result = get_agent_variable(agent, variable_name)
```

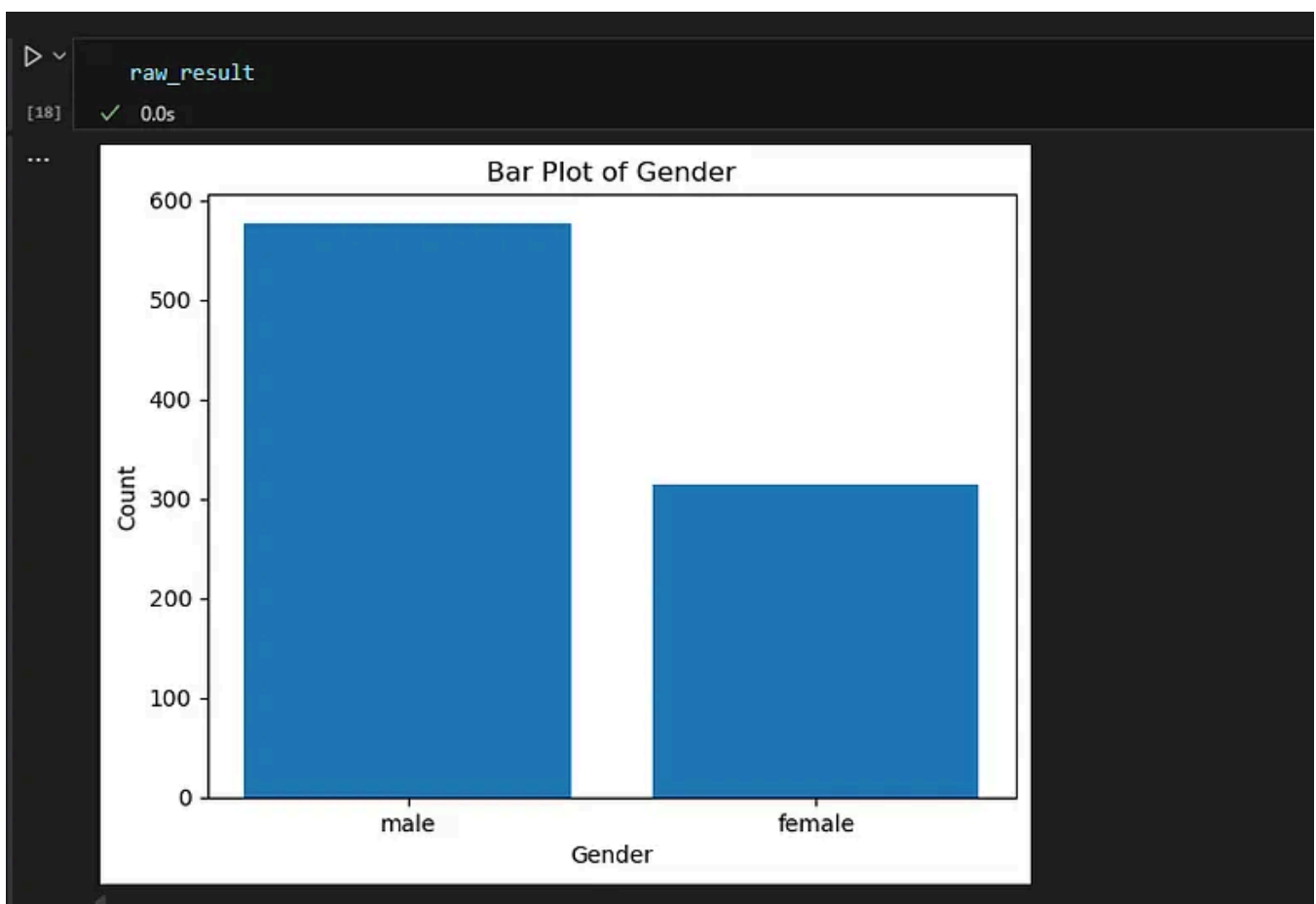


Image by Author

4.4 OutputParser errors

If you re-run the same `agent_invoke` code again, you may encounter

```

# Invoke the agent
agent_response = agent.invoke("input": final_prompt))

...
OutputParserException: Traceback (most recent call last)
File c:\Users\Prateek's Pad\conda\envs\da_env2\Lib\site-packages\langchain\agents\agent.py:1352, in AgentExecutor._iter_next_step(self, name_to_tool_map, color_map, ...
1351     # Call the LLM to see what to do.
-> 1352     output = self.action_agent.plan(
1353         intermediate_steps,
1354         callbacks=run_manager.get_child() if run_manager else None,
1355         **inputs,
1356     )
1357 except OutputParserException as e:

```

Image by Author

OutputParserException errors occur when an experimental LangChain agent's LLM output doesn't match the parser's required schema, causing the agent to fail mid-run instead of producing an action or final answer.

What it is

- An OutputParserException is a specific exception LangChain raises when a parser (e.g., ReAct/MRKL style agent output parser) cannot parse the model's text into a valid Action/Action Input or Final Answer object.
- It's common in experimental agents like `pandas_dataframe_agent` because they rely on strict ReAct formatting and tool invocation (e.g., `python_repl_ast`) that LLMs occasionally violate under realistic prompts and data.

Why it happens

- The LLM returns plain prose or partial "Thought" without the mandated "Action:" and "Action Input:" blocks, breaking the regex-based parser expectations for ReAct/MRKL.
- The LLM outputs both a parseable action and a Final Answer in the same turn, which some parsers explicitly treat as invalid and raise an exception.
- Prompt templates or system messages inadvertently add tokens like "Input:" that conflict with the parser's expected slots, or the LLM ignores formatting instructions under complex tasks or certain models/settings.

Issues have been observed across agents (pandas, SQL, general ReAct), across different LLMs, and across versions; some library updates mitigated specific cases, but the fundamental brittleness of free-form generations against strict parsers remains a known limitation.

- We resolve this through langchain's OutputParserException. Older argument of `handling_parsing_errors` has been depreciated. We add these exceptions within

the code

- Reduce ambiguity and keep the prompt simple and clear with instruction requires exactly one Final Answer
- If a model frequently violates formatting, try a retry policy with a corrective message. This is achieved through `max_retries` argument in the agent

Now lets bring it all together with some improved exception handling and logging embedded for more consistent and higher success in generating output as per user's query

5. Bringing it all together

- Set up logs and warnings
- Set up your `env`
- Define upgraded helper functions
- Final Agent function with prompt classification, final prompt structure, defining the agent and retry mechanism with custom output parser

```
# =====
# Loading key packages
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import langchain as lc
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
import anyio
from claude_code_sdk import query
from langchain_openai import OpenAI
from langchain_experimental.agents import create_pandas_dataframe_agent
import seaborn as sns
from langchain.schema import SystemMessage, HumanMessage
import logging
from typing import Any, Optional
from langchain_experimental.agents.agent_toolkits import create_pandas_dataframe_agent
import warnings
import io
import base64
from langchain.output_parsers import OutputFixingParser
```

```

from langchain_core.exceptions import OutputParserException
from langchain_core.output_parsers import BaseOutputParser
import time

# =====
# Loggings and Warnings
# =====

# Set up basic logging
logging.basicConfig(
    level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s"
)

# Suppress INFO logs from httpx and httpcore
logging.getLogger("httpx").setLevel(logging.WARNING)
logging.getLogger("httpcore").setLevel(logging.WARNING)

# Suppress FutureWarning messages
warnings.simplefilter(action="ignore", category=FutureWarning)

# =====
# Env set up
# =====

load_dotenv()
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = os.getenv("OPENROUTER_BASE_URL")
ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY")

# =====
# Helper functions and classes
# =====

# Custom Output Parser for Agent Results
class AgentResultParser(BaseOutputParser):
    """Custom parser to handle agent execution results"""

    def parse(self, text: str) -> str:
        """Parse the agent output text"""
        if not text or text.strip() == "":
            raise OutputParserException("Empty or None output received from agent")
        return text.strip()

    @property
    def _type(self) -> str:
        return "agent_result_parser"

```

```

def get_agent_variable(agent: Any, variable_name: str) -> Optional[Any]:
    """
    Retrieves a variable from the agent's Python tool local execution environment
    """
    try:
        python_tool = agent.tools[0]
        return getattr(python_tool, "locals", {}).get(variable_name)
    except (IndexError, AttributeError) as e:
        logging.error(
            f"Could not access tools or local variables from the agent. Details: {e}"
        )
    return None


def format_output_response(result_type: str, raw_result: Any, user_query: str) -> dict:
    """
    Format the raw result into a consistent response structure for the frontend
    """
    try:
        if raw_result is None:
            return {
                "status": "error",
                "message": f"No result generated for query: {user_query}",
                "data": {
                    "error": f"The agent did not produce any {result_type} result"
                },
                "result_type": result_type,
            }

        if result_type == "statistic":
            return {
                "status": "success",
                "message": f"Statistical analysis complete: {raw_result}",
                "data": {
                    "value": raw_result,
                    "type": "scalar",
                    "description": f"Result for: {user_query}",
                },
                "result_type": "statistic",
            }

        elif result_type == "table":
            if hasattr(raw_result, "to_dict"):
                return {
                    "status": "success",
                    "message": f>Data table generated successfully with {len(raw_result.columns)} columns",
                    "data": {
                        "dataframe": raw_result,
                        "type": "dataframe",
                        "shape": raw_result.shape,
                        "columns": list(raw_result.columns),
                        "description": f"Table result for: {user_query}",
                    },
                }
    except Exception as e:
        logging.error(f"Error in format_output_response: {e}")
    return {}

```

```

        },
        "result_type": "table",
    }
else:
    return {
        "status": "error",
        "message": "Expected a DataFrame but received a different c
        "data": {
            "error": f"Received {type(raw_result)} instead of DataF
        },
        "result_type": "table",
    }

elif result_type == "graph":
    if raw_result is not None:
        if hasattr(raw_result, "savefig"):
            return {
                "status": "success",
                "message": "Visualization generated successfully",
                "data": {
                    "figure": raw_result,
                    "type": "matplotlib",
                    "description": f"Graph result for: {user_query}",
                },
                "result_type": "graph",
            }
        elif hasattr(raw_result, "show") and hasattr(raw_result, "data")
            return {
                "status": "success",
                "message": "Interactive visualization generated success
                "data": {
                    "figure": raw_result,
                    "type": "plotly",
                    "description": f"Interactive graph result for: {use
                },
                "result_type": "graph",
            }
        else:
            return {
                "status": "error",
                "message": "Generated figure is not in a recognized for
                "data": {
                    "error": f"Figure type {type(raw_result)} not supp
                },
                "result_type": "graph",
            }
    else:
        return {
            "status": "error",
            "message": "No visualization was generated",
            "data": {"error": "The agent did not produce a figure"},
            "result_type": "graph",
        }

```

```

        else:
            return {
                "status": "error",
                "message": f"Unknown result type: {result_type}",
                "data": {"error": f"Unsupported result type: {result_type}"},
                "result_type": result_type,
            }

    except Exception as e:
        logging.error(f"Error formatting output: {e}")
        return {
            "status": "error",
            "message": f"Error processing {result_type} result: {str(e)}",
            "data": {"error": str(e)},
            "result_type": result_type,
        }

# =====
# Main Function
# =====

def agent_at_work(model: Any, user_query: str, path: Any) -> dict:
    """
    Invokes a pandas dataframe agent to process a user query, classifying the e
    and returning a formatted result for the frontend.
    Enhanced with OutputFixingParser for robust error handling.
    """

    # =====
    # # Define classification prompt as a constant
    # =====

    CLASSIFICATION_PROMPT = """
    You are a classifier. Read the user request carefully and decide what type
    There are ONLY three possible categories: `statistic`, `table`, or `graph`.
    RULES:
    - The answer MUST be strictly one word: either `statistic`, `table`, or `gr
    - Do not explain. Do not output anything else.
    - Example: User request: "generate a bar plot of gender" -> Answer: graph
    - Example: User request: "show me the correlation matrix" -> Answer: table
    - Example: User request: "what is the average age" -> Answer: statistic
    """

    try:

        # =====
        # Step 0: Define LLM
        # =====
        def create_chat_openai(model):
            """Create a ChatOpenAI instance with a specified model."""

```



```

try:
    llm = ChatOpenAI(
        model=model,
        openai_api_key=OPENROUTER_API_KEY,
        openai_api_base=OPENROUTER_BASE_URL,
        streaming=False,
        verbose=False,
        temperature=0.0,
        max_retries=3,
    )
    return llm
except Exception as e:
    logging.error(f"Failed to create ChatOpenAI instance: {e}")
    return None

llm = create_chat_openai(model)
if llm is None:
    return {
        "status": "error",
        "message": "Failed to initialize the language model",
        "data": {"error": "LLM initialization failed"},
        "result_type": "unknown",
    }

# =====
# Step 1: Create custom output parser and wrap it with OutputFixingParser
# =====

base_parser = AgentResultParser()
output_fixing_parser = OutputFixingParser.from_llm(parser=base_parser,

#
# =====
# Step 2: Classify the user query with error handling
# =====
try:
    classification_result = llm.invoke(
        CLASSIFICATION_PROMPT + f"\nUser request: '{user_query}'"
    )
    prompt_type = classification_result.content.strip().lower()

    # NEW: Use OutputFixingParser to ensure valid classification
    try:
        fixed_classification = output_fixing_parser.parse(prompt_type)
        prompt_type = fixed_classification.lower()
    except OutputParserException as parse_error:
        logging.warning(
            f"Classification parsing failed: {parse_error}. Using fallback"
        )
        prompt_type = "table" # Fallback to table

except Exception as e:
    logging.error(f"LLM classification failed: {e}")

```

```

        return {
            "status": "error",
            "message": "Failed to classify the user query",
            "data": {"error": str(e)},
            "result_type": "unknown",
        }

valid_types = {"statistic", "table", "graph"}
if prompt_type not in valid_types:
    logging.warning(
        f"Invalid classification type returned: '{prompt_type}'. Default
    )
    prompt_type = "table"

# =====
# Step 3: Formulate the final prompt based on the classification
# =====
prompt_map = {
    "statistic": f"{user_query} Assign the final result value to a vari
    "table": f"{user_query} Assign the output dataframe to a variable r
    "graph": f"{user_query} Create the visualization and assign the fig
}
final_prompt = prompt_map.get(prompt_type, user_query)

# Load the dataframe
try:
    df = pd.read_csv(path)
except Exception as e:
    return {
        "status": "error",
        "message": f"Failed to load CSV file: {str(e)}",
        "data": {"error": f"File loading error: {str(e)}"},
        "result_type": prompt_type,
    }

#
# =====
# Step 4: Create the agent
# =====
try:
    agent = create_pandas_dataframe_agent(
        llm,
        df,
        verbose=False,
        allow_dangerous_code=True,
        return_intermediate_steps=False,
    )
except Exception as e:
    logging.error(f"Agent creation failed: {e}")
    return {
        "status": "error",
        "message": f"Failed to create the analysis agent: {str(e)}",
        "data": {"error": str(e)},
    }

```

```

        "result_type": prompt_type,
    }

# =====
# Step 5: Enhanced agent invocation with retry mechanism
# =====

max_retries = 10 # can be adjusted but in my experience , successful r
retry_count = 0
raw_result = None

while retry_count < max_retries and raw_result is None:
    try:
        # Invoke the agent
        agent_response = agent.invoke({"input": final_prompt})

        # NEW: Apply OutputFixingParser to the agent response if it's a
        if isinstance(agent_response, str):
            try:
                fixed_response = output_fixing_parser.parse(agent_respo
                logging.info(
                    f"Agent response fixed by OutputFixingParser: {fixe
                )
            except OutputParserException as parse_error:
                logging.warning(
                    f"OutputFixingParser failed: {parse_error}. Using c
                )

            # Retrieve the result from agent's local variables
            variable_map = {
                "statistic": "sg_value",
                "table": "temp_df",
                "graph": "figure",
            }
            variable_name = variable_map.get(prompt_type)
            raw_result = get_agent_variable(agent, variable_name)

        # NEW: If result is still None, wait and retry
        if raw_result is None:
            retry_count += 1
            if retry_count < max_retries:
                logging.warning(
                    f"Attempt {retry_count} failed to get result. Retry
                )
                time.sleep(1) # Brief pause before retry
            else:
                logging.error(
                    f"Failed to get result after {max_retries} attempts
                )

    except OutputParserException as parse_error:
        logging.error(
            f"Output parsing error on attempt {retry_count + 1}: {parse

```

```

    )
    retry_count += 1
    if retry_count < max_retries:
        logging.info(f"Retrying agent invocation due to parsing error")
        time.sleep(1)
    else:
        return {
            "status": "error",
            "message": f"Output parsing failed after {max_retries} attempts",
            "data": {"error": str(parse_error)},
            "result_type": prompt_type,
        }

except Exception as e:
    logging.error(f"Agent invocation failed on attempt {retry_count + 1}: {e}")
    retry_count += 1
    if retry_count < max_retries:
        logging.info(f"Retrying agent invocation due to general error")
        time.sleep(1)
    else:
        return {
            "status": "error",
            "message": f"The agent could not process the query after {max_retries} attempts",
            "data": {"error": str(e)},
            "result_type": prompt_type,
        }

# =====
# Step 5: Final validation and formatting & last attempts
# =====

if raw_result is None:
    # Last attempt with OutputFixingParser to generate a meaningful error message
    try:
        error_prompt = f"The agent failed to generate {prompt_type} for {query}. Please provide a valid response."
        error_response = llm.invoke(error_prompt)
        fixed_error = output_fixing_parser.parse(error_response.content)
        return {
            "status": "error",
            "message": f"No result generated after all retry attempts: {fixed_error}",
            "data": {"error": "Maximum retry attempts exceeded"},
            "result_type": prompt_type,
        }
    except:
        return {
            "status": "error",
            "message": f"No result generated after {max_retries} retry attempts",
            "data": {"error": "Maximum retry attempts exceeded"},
            "result_type": prompt_type,
        }

```

```
# Format and return the response
return format_output_response(prompt_type, raw_result, user_query)

except Exception as e:
    logging.error(f"Unexpected error in agent_at_work: {e}")
    return {
        "status": "error",
        "message": f"An unexpected error occurred: {str(e)}",
        "data": {"error": str(e)},
        "result_type": "unknown",
    }
```

Testing some examples

- bar plot of class

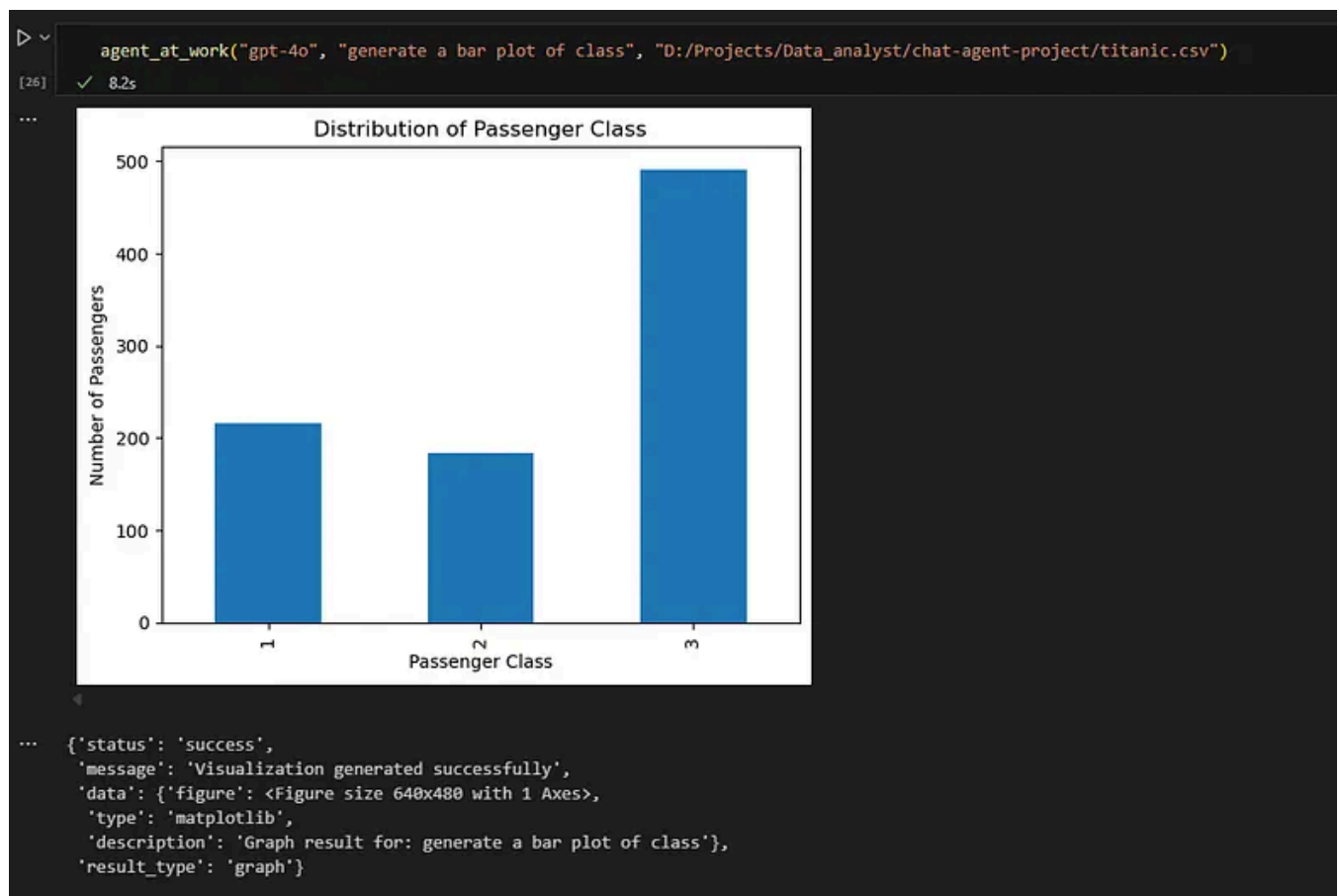


Image by Author

- Overlaying density plot of age by class with average dotted lines built in seaborn



Image by Author

- Logistic regression model build and show coefficients

```

agent_at_work("gpt-4o", "Build a logistic regression model to predict survival based on age, fare, and class. Show me the model summary and coefficients.", "D:/Projects/Data_analyst/chat-agent-project/titanic.csv")

[ ] ✓ 224s

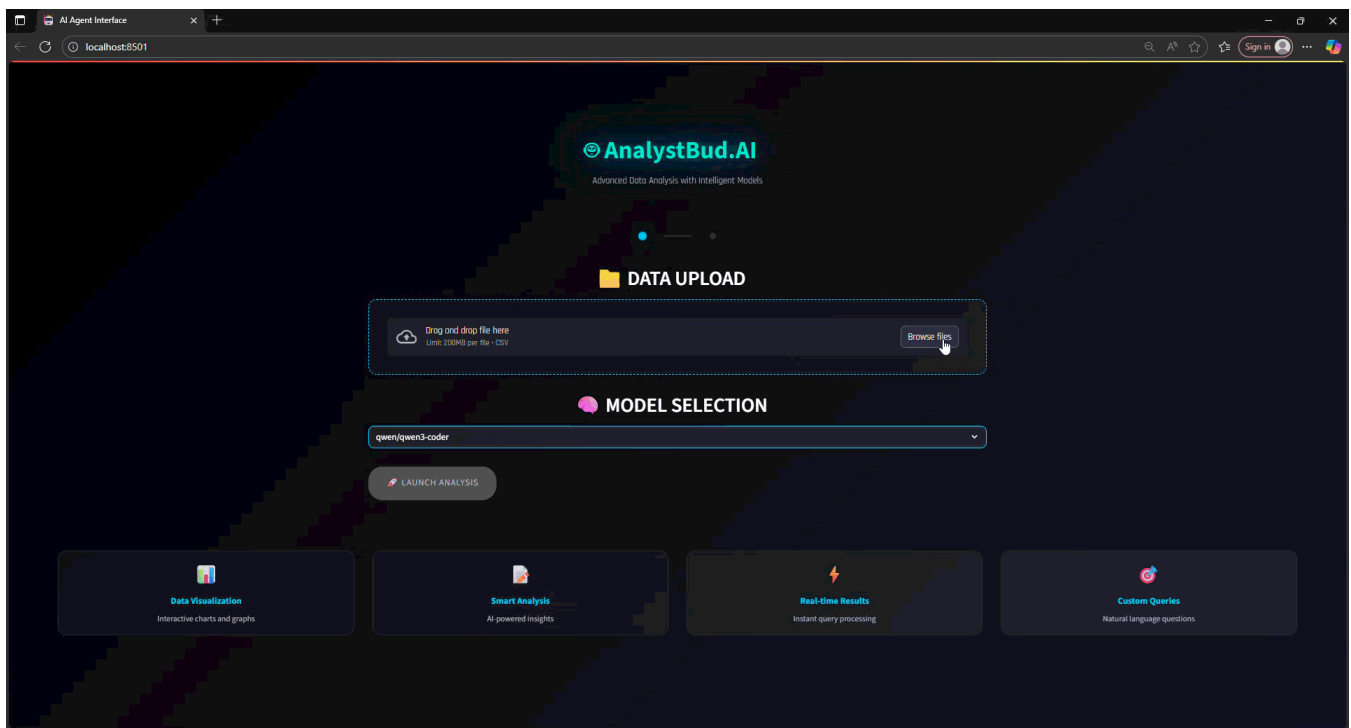
{
  "status": "success",
  "message": "Data table generated successfully with 3 rows and 2 columns",
  "data": {
    "dataframe":
      Feature    Coefficient
0    Age    -0.381453
1    Fare     0.194220
2    Pclass -0.718577,
    "type": "dataframe",
    "shape": (3, 2),
    "columns": ['Feature', 'Coefficient'],
    "description": "Table result for: Build a logistic regression model to predict survival based on age, fare, and class. Show me the model summary and coefficients.",
    "result_type": "table"
  }
}

```

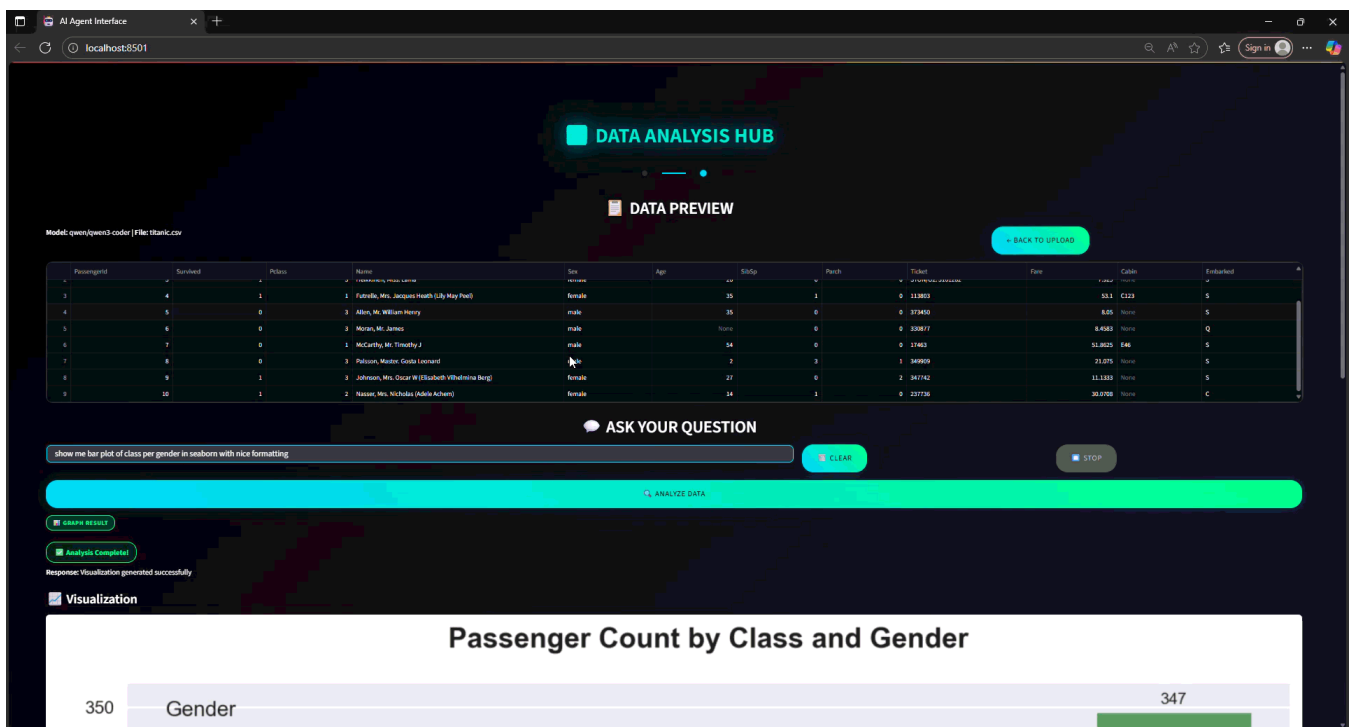
Image by Author

5. Streamlit App to give a UI touch

You can now build a simple dark theme front end with streamlit which interacts with `agent_at_work` in backend. You can allow user to select from a choice of models along with the dataframe while allowing basic exporting functionality for tables, graphs etc.



Landing page showing an example of output with user query



Data analysis hub

The app UI code can be found [here](#).

6. Learnings and future enhancement areas

- For greater control, security and customization — build your own agent from scratch in a sandboxed Python REPL tool. Integrate other tools of choice to

enhance functionality. For quick prototypes, start with specialized, purpose-built agents rather than generic ones.

- Identify common failure pattern and deal with Output parser , well defined prompts and specifying Agent's behavior around "Thought → Action → Action Input → Observation ... → Final Answer," to manage Schema Violations
- Build resilience in codes to handle any type of unexpected outputs and always have defaults defined
- Spend time choosing the right coding model, assess inference and cost.
- cache and memory can be the next two components in the architecture to improve the tool's usability
- LangGraph Integration: For more complex multi-step analysis workflows, Integration with visualization libraries like Plotly, statistical packages
- Model Orchestration: Dynamic model selection based on query complexity

References

1. https://python.langchain.com/api_reference/openai/chat_models/langchain_openai.chat_models.base.ChatOpenAI.html
2. <https://github.com/hwchase17/langchain/issues/3303>
3. https://python.langchain.com/api_reference/core/exceptions/langchain_core.exceptions.OutputParserException.html
4. <https://python.langchain.com/docs/integrations/tools/pandas/>
5. <https://stackoverflow.com/questions/76421921/using-gpt-4-or-gpt-3-5-with-sql-database-agent-throws-outputparserexception-cou>
6. https://python.langchain.com/api_reference/core/exceptions/langchain_core.exceptions.OutputParserException.html

[Langchain](#)[Llm Applications](#)[Ai Agents In Action](#)



Following

Published in Data Science Collective

934K followers · Last published 7 hours ago

Advice, insights, and ideas from the Medium data science community



Edit profile

Written by Prateek Bhatnagar

175 followers · 43 following

Data Geek, Short story writer

